

Лабораторна робота 6. Проектування архітектури онлайн-магазину в AWS

Мета: Спроекувати базову cloud-архітектуру онлайн-магазину в AWS, зобразити її на діаграмі та обґрунтувати ключові архітектурні рішення.

Сценарій

Компанія запускає онлайн-магазин ноутбуків та аксесуарів. Бізнес хоче розгорнути систему в AWS. Команда розробки вже визначилась з основною функціональністю, але перед реалізацією потрібно підготувати архітектурне рішення.

Функціональність системи

Система має підтримувати:

- публічний каталог товарів;
- зображення товарів;
- пошук і фільтрацію товарів;
- реєстрацію та логін користувачів;
- кошик;
- оформлення замовлення;
- оплату через зовнішнього платіжного провайдера;
- email-підтвердження замовлення;
- особистий кабінет користувача з історією замовлень;
- адмін-панель для керування товарами й замовленнями.

Очікуване навантаження

- Каталог: приблизно 1000–5000 SKU.
- Очікуване зростання каталогу: у 2–3 рази за 2 роки.
- Зареєстровані користувачі: приблизно 10 000-30000.
- Звичайне навантаження: 50-300 одночасних користувачів.
- Основний тип навантаження: перегляд каталогу.
- Піки: до 2000-5000 одночасних користувачів під час акцій.
- Пікові замовлення: до 500-700 замовлень за годину протягом кількох годин.
- Географія користувачів: переважно Україна та сусідні країни ЄС.

Основне навантаження створює read-heavy частина системи: каталог, сторінки товарів і зображення.

Checkout має менший обсяг запитів, але вищу критичність: підтверджене замовлення не має бути втрачене, а повторні запити від користувача або платіжного провайдера не мають створювати дублікати.

Нефункціональні вимоги

Архітектура має враховувати такі вимоги:

- Availability: 99,9% для основної клієнтської частини системи.
- Catalog latency: p95 завантаження сторінки каталогу < 500 мс для основної географії користувачів.
- Checkout latency: p95 синхронної частини оформлення замовлення < 2 с.
- Durability замовлень: жодне підтверджене замовлення не може бути втрачене.
- Scalability: система має витримувати пікове навантаження до $\times 10\text{--}20$ звичайного.
- Cost: орієнтовний бюджет у звичайному режимі – \$1000–2 000 на місяць.

Обмеження

- Платежі виконуються через зовнішнього провайдера, наприклад LiqPay або Stripe.
- Система не зберігає номери банківських карток і не обробляє платіжні дані напряду.
- Дані користувачів і замовлень мають зберігатися безпечно: доступ лише для авторизованих компонентів, шифрування даних у сховищах, резервне копіювання та аудит критичних операцій.
- AWS-регіон потрібно обрати з урахуванням географії користувачів, latency, вартості та доступності потрібних сервісів.

Завдання 1. Архітектурна діаграма

Створіть архітектурну діаграму в draw.io / diagrams.net або іншому зручному інструменті з використанням наборів іконок конкретних сервісів. Додайте зображення в звіт.

У draw.io відкрийте панель бібліотек фігур: More Shapes... / + More Shapes, увімкніть набір AWS Architecture або скористайтесь AWS-шаблоном при створенні нового файлу.

Можна також використати інший інструмент для архітектурних діаграм, якщо він дозволяє зрозуміло показати AWS-компоненти, зв'язки між ними, VPC, Availability Zones і зовнішні системи.

На діаграмі потрібно показати:

- AWS Region;
- мінімум 2 Availability Zones;
- VPC;
- public/private subnets;
- CloudFront;
- S3 для статички, зображень товарів або обох варіантів;
- ALB;
- compute layer: EC2 ASG, ECS або Lambda;
- базу даних для користувачів, товарів і замовлень;
- кеш, якщо ви його використовуєте;
- чергу або topic для асинхронної обробки, якщо ви її використовуєте;
- базові компоненти моніторингу: логи, метрики, алерти.

На діаграмі також покажіть зовнішні системи, з якими взаємодіє застосунок (платіжні шлюзи, поштові провайдери).

На діаграмі мають бути зрозумілі основні потоки:

1. Користувач відкриває сторінку товару.
Покажіть шлях запиту до статички, зображень, API, кешу та бази даних.
2. Користувач оформлює замовлення.
Покажіть шлях від frontend/API до бази замовлень і платіжного провайдера.
3. Платіж підтверджено.
Покажіть, як система отримує підтвердження оплати та оновлює статус замовлення.
4. Email-повідомлення.
Покажіть, чи надсилається email синхронно або через асинхронну чергу.

Завдання 2. Обґрунтування ключових рішень

У короткому документі опишіть 5–6 ключових архітектурних рішень.

Не потрібно обґрунтовувати кожен сервіс на діаграмі. Достатньо пояснити найважливіші рішення, від яких залежить архітектура системи.

Для кожного рішення використайте такий формат:

Рішення: що обрали.

Чому: яку вимогу це закриває.

Альтернатива: що можна було обрати замість цього.

Trade-off: що отримуємо і чим платимо.

Приклади рішень, які можна обґрунтувати:

- CloudFront + S3 для статички й зображень.
- ALB + ECS/EC2 для backend API.
- RDS Multi-AZ для користувачів, товарів і замовлень.
- DynamoDB для частини даних, якщо ви обрали NoSQL-підхід.
- ElastiCache для каталогу, сесій або гарячих даних.
- SQS/SNS для email-нотифікацій або асинхронної обробки замовлення.
- Вибір AWS-регіону.
- Вибір між EC2, ECS, Fargate і Lambda.
- Відмова від Multi-Region через бюджет і складність.

Приклад обґрунтування:

Рішення: CloudFront + S3 для зображень товарів.

Чому: каталог і зображення читаються значно частіше, ніж оновлюються. CloudFront дозволяє віддавати зображення з edge-локацій ближче до користувача і зменшує навантаження на backend.

Альтернатива: віддавати зображення напряму з backend через ALB.

Trade-off: CloudFront зменшує latency і навантаження на backend, але додає проблему інвалідації кешу при оновленні зображень.

Частина 3. Поведінка при збоях

Коротко опишіть, як ваша архітектура поводить себе у трьох ситуаціях.

3.1. Впав один backend instance

Поясніть:

- що бачить користувач;
- чи продовжує система працювати;
- яку роль відіграють ALB, health checks і Auto Scaling;

- чи втрачаються дані.

3.2. Впав primary RDS instance

Поясніть:

- що відбувається з checkout у цей момент;
- чи буде коротка недоступність бази;
- як працює failover;
- чи можуть загубитися підтверджені замовлення;
- які механізми потрібні для захисту: транзакції, retry, idempotency, backups.

3.3. Платіжний провайдер недоступний 10 хвилин

Поясніть:

- що бачить користувач при спробі оплатити;
- чи створюється замовлення;
- який статус отримує замовлення;
- чи потрібні retry;
- як уникнути подвійної оплати або подвійного створення замовлення;
- яку роль відіграють idempotency keys і webhook-и.

Список літератури:

1. <https://aws.amazon.com/architecture/icons/>
2. <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>
3. <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-example-private-subnets-nat.html>
4. <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>
5. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html>
6. <https://calculator.aws/>